



COMP47350: Data Analytics (Conv)

Dr. Georgiana Ifrim

georgiana.ifrim@ucd.ie

Insight Centre for Data Analytics

ML-Labs Centre for Research Training in Machine Learning

School of Computer Science

University College Dublin

2025/26

Module Topics

- **Python Environment** (Anaconda, Jupyter Notebook)
- **Getting Data** (Web scraping, APIs)
- **Understanding Data** (slicing, visualisation)
- **Preparing Data** (cleaning, transformation)
- **Modeling & Evaluation** (machine learning)

Getting Data (Web scraping, APIs)

- Data is generated in different domains (climate, engineering, finance) and stored in different formats
- **This lecture: Hands-on examples of getting and working with data in different formats**
 - Web scraping (TXT, CSV, JSON, HTML, XML, PDF)
 - APIs (JSON)
- Most of the time we work with **data in csv format**; we discuss some ways to get to a csv format from other data formats

Getting Data: Considerations

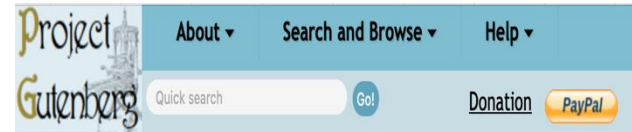
What kind of things should you think about when you retrieve data?

- **Accessibility:** Is the data publicly accessible? Is an account required for authentication (get developer login or tokens)? Do specific terms of service apply?
- **Ethics:** Are there any privacy or ethical considerations in using, aggregating or republishing the data? Are you working with any identifiable personal data? Are you familiar with GDPR legal requirements (or can consult someone who is)?
- **Scale:** How much data is available? Is historic/past data available? Is the data delivered in batches or streaming in real-time? What are the storage requirements (eg SQL or noSQL database, json, csv files)?
- **Pre-processing:** What is the format of the data (storage standards)? What automated pre-processing techniques should be applied? Does the data contain missing values? Is data cleaning required?
- **Verification:** Is the data reliable or potentially inaccurate, misleading or biased? Is automated or manual curation required? What are some useful **data integrity checks?** I.e., like unit tests for checking data, e.g., check that a feature expected to only have positive values, does not have negative values.

Web Scraping

- **Download a file from a given URL**
- Example using Python package **requests** to download a text file (.txt) from the Project Gutenberg website

```
#Get a text file.  
#Get book "Alice's Adventures in Wonderland" from Project Gutenberg,  
import requests  
url='http://www.gutenberg.org/cache/epub/11/pg11.txt'  
text_page = requests.get(url).text  
#print(text_page)  
  
#Look at the first 500 characters of the book  
print(text_page[:500])
```



Welcome to Project Gutenberg

Project Gutenberg is a library of over 70,000 free eBooks: <https://www.gutenberg.org>

Data Format: .txt

- **Text format (.txt):** Can work directly on the data (no specific markup to explain the file structure)
- Many good Python libraries to work with text data (e.g., `nltk`)
- What you see if you look at the file with a simple text editor or with the Terminal

```
Project Gutenberg's Alice's Adventures in Wonderland, by Lewis Carroll
```

```
This eBook is for the use of anyone anywhere at no cost and with  
almost no restrictions whatsoever. You may copy it, give it away or  
re-use it under the terms of the Project Gutenberg License included  
with this eBook or online at www.gutenberg.org
```

```
Title: Alice's Adventures in Wonderland
```

```
Author: Lewis Carroll
```

```
Posting Date: June 25, 2008 [EBook #11]
```

```
Release Date: March, 1994
```

```
[Last updated: December 20, 2011]
```

Data Format: .csv

- **CSV format (.csv)** “Comma Separated Values”: Text file formatted in columns. Lots of good Python libraries to work with .csv (e.g., Python package **pandas**)
- What you see if you look at the file with a text editor
Name, Weight, Height
Alice, 60, 1.56
Bob, 72, 1.75
David, 84, 1.82
- First row (the header) explains what the columns mean
- Each **example is on a row**, each **column is a feature** describing the example
- **We bring our data into this format for further data analysis**

Data Format: .csv

- Read a CSV file into a Data Frame object
- Example using Python package **pandas** (uses a data structure called a **dataframe** to store data)

```
# Import package for reading csv files
import pandas as pd

# Reading from a csv file, into a data frame
df = pd.read_csv('MotorInsuranceFraudClaimABTFull.csv')

# Show the data frame
df

# Show first 10 rows of data frame
# The rows are indexed starting from 0
#df.head(10)
```

	ID	Insurance Type	Income of Policy Holder	Marital Status	Num Claimants	Injury Type	Overnight Hospital Stay	Claim Amount	Total Claimed	Num Claims	Num Soft Tissue	% Soft Tissue	Claim Amount Received	Fraud Flag
0	1	CI	0	NaN	2	Soft Tissue	No	1625	3250	2	2.0	1.0	0	1
1	2	CI	0	NaN	2	Back	Yes	15028	60112	1	0.0	0.0	15028	0
2	3	CI	54613	Married	1	Broken Limb	No	-99999	0	0	0.0	0.0	572	0
3	4	CI	0	NaN	3	Serious	Yes	270200	0	0	0.0	0.0	270200	0

Data Format: .json

- **JSON (JavaScript Object Notation):** a **lightweight text format** popular for online data exchange. Based originally on the JavaScript language and (relatively) **easy for humans to read and write** - <http://www.json.org>
- JSON files are built from two different structures:
 1. **Object:** Collection of name/value pairs - **like a Python dictionary.**
Begins with { and ends with }
Each name is followed by : (colon) and the name/value pairs are separated by commas
 2. **Array:** A list of values separated by commas - **like a Python list.**
Begins with [and ends with]
- Values can be of the following types: string in double quotes; a number; true/false; null; object; array.

Data Format: .json

- **JSON:** Text file with special format (dictionary-style) used to transmit objects consisting of **attribute-value pairs**
- [Competing with the XML format](#) for transmitting data
- Example **json**

```
{  
  "firstName": "John",  
  "lastName": "Smith",  
  "age": 25,  
  "address": {  
    "street": "21 Street",  
    "city": "New York",  
  },  
  "children": [],  
  "married": false,  
  "spouse": null  
}
```

Data Format: .json

Python library to parse JSON data (package **json**)

```
#Library to parse json objects
import json
json_data = json.load(open('example.json'))
#json_data looks like a nested Python dictionary
print(json_data)
```

```
#We can refer to different fields of the json object
print(json_data['glossary']['title'])
print(json_data['glossary']['GlossDiv']['title'])
print(json_data['glossary']['GlossDiv']['GlossList']['GlossEntry']['ID'])
```

```
{
  "glossary": {
    "title": "example glossary",
    "GlossDiv": {
      "title": "S",
      "GlossList": {
        "GlossEntry": {
          "ID": "SGML",
          "SortAs": "SGML",
          "GlossTerm": "Standard Generalized Markup Language",
          "Acronym": "SGML",
          "Abbrev": "ISO 8879:1986",
          "GlossDef": {
            "para": "A meta-markup language, used to create markup languages such as DocBook.",
            "GlossSeeAlso": ["GML", "XML"]
          },
          "GlossSee": "markup"
        }
      }
    }
  }
}
```

Data Format: .html, .xml

- **HTML, XML:** Markup Languages. We need a parsing library to extract the text for given tag names. Python packages for HTML: **BeautifulSoup**, **newspaper**, packages for XML: **feedparser**.
- **XML describes data. HTML describes data and its presentation** (e.g., font styles, colours). RSS newsfeeds are often published as XML files, e.g., <http://www.rte.ie/news/rss/business-headlines.xml>
- Example
HTML:

```
<link rel="canonical" href="http://www.irishtimes.com/news/science/evidence-of-ne-  
solar-system-1.2505306" />  
<title> Evidence of new planet '10 times size of Earth' in solar system </title>  
<meta name="description" content="Existence of 'Planet Nine' shown by examin  
r bodies " />  
  
<meta property="vf:unique_id" content="www.irishtimes.com12505306" />  
  
<meta itemprop="datePublished" content="2016-01-21T10:12:50+0000"/>  
  
<meta name="news_keywords" content="News, Science, Astronomy & Space, Konstantin  
stitute Of Technology, Kuiper" />
```
- Need to look at the structure and content of tags to know where the useful information is. We can open the html file with a text editor or in the browser to see the html structure.

Data Format: .html, .xml

- Example of reading/parsing an html file using Python package **BeautifulSoup** (another useful and easier to use library is **newspaper**)

```
#Use package 'beautifulsoup' to extract the content of HTML fields
#Need to know the HTML structure and the tags containing the information we need
#To look at the HTML file open it in a text editor, look for the tags that contain headline
#If you don't have beautifulsoup4 installed, run in shell: conda install beautifulsoup4
from bs4 import BeautifulSoup

# Method to parse the structure of an html page using package beautifulsoup.
# The code looks for specific tags in the html structure and extracts the content
def getArticleDetailsByUrl(url):
    page = requests.get(url)
    soup = BeautifulSoup(page.text, "html.parser")
    #soup.prettify()

    headline = soup.title.string
    subheadline = soup.head.find("meta", attrs={"name": "description"}).get('content')
```

Data Format: .pdf

- **PDF or DOC:** These are special proprietary data formats; we need tools to extract text from the binary encoding
- **Many reports are published in this format (eg company earnings, government policy, health reports)**
- We will use a Python library named **pdftotext**
pip install pdftotext
- If you have difficulty installing this library, check specific information for your operating system: <https://pypi.org/project/pdftotext/>

Data Format: .pdf

- What you see if you look at a PDF with a text editor

```
b'%PDF-1.5\n%\xd0\xd4\xc5\xd8\n2 0 obj <<\n/Type /ObjStm\n/N 100\n/e\n>>\nstream\nx\xda\x9dV\xdbn\x9cH\x10}\x9f\xaf\xa8\xc7d\xb5\xa8\ne8\x19\xa30\x80\x1a\xb0=\xfb\xfb{\xa8\xb\xed\xec\xa5\x07\xed\x83M\90P$dFB\x930\x92\x84%\x91c\x99\x91\x14)\xfeHj\xbb\x93\x92\xa4\xcl\>x9ek\xd2\xd2\x92\xb2\xa4S<\xcfH\xdb\x84\xf3\xa5\x89\xdcI)\xb2\xe8\>a7\xd4\xa2J`3\xb2\xa00\x82\xacU;\xd4h\xf3\x8cLJ\x19R\x1bC\x19\x12\>6\x00\x84F\x05P\xe81Q\x922n\xdc\xe8j\x86f\x13\x0b\x9a\x94\x84H2\xce&9a)\xa4\xc2\x15|2e2\xc1\x02\xaa\x9dH\xc0\xc8\x02\x89\x04\x94J1?\xc7\x01b\x9d\xe1\x1f\xc4\x17)\x07B-\x91j\xb9\x130@\xa4h\x19\t\xb1\x98r\t\x98!\xac\xe6Z\x10g\xb9\x7f\x18"l\xe;\x9c8\x93\xd9N\xc0\x14\x91\x10\x00\xacPE&\xdc\x13\x0c\x92\xc9T\xa6\xc5b\xa8\xclx$<'
```

- The text extracted from the PDF using **pdftotext** (first 500 characters)

```
b'Think Python\nHow to Think Like a Computer Scientist\nVersion 2.0.17\n\n\nx(omputer Scientist\nVersion 2.0.17\nAllen Downey Green Tea Press\nNeedham, Maslen Downey.\nGreen Tea Press 9 Washburn Ave Needham MA 02492 Permission is grthis document under the terms of the Creative Commons Attribution-NonCommerci\nble at http: //creativecommons.org/licenses/by-nc/3.0/.\n'
```

Data Format: .pdf

- Example using **pdftotext** to extract text from a pdf file named “thinkpython.pdf”

```
import pdftotext

# Make sure you have downloaded the "thinkpython.pdf" file in your current folder
# http://www.greenteapress.com/thinkpython/thinkpython.pdf

# Load your PDF
with open('thinkpython.pdf', 'rb') as f:
    pdf = pdftotext.PDF(f)

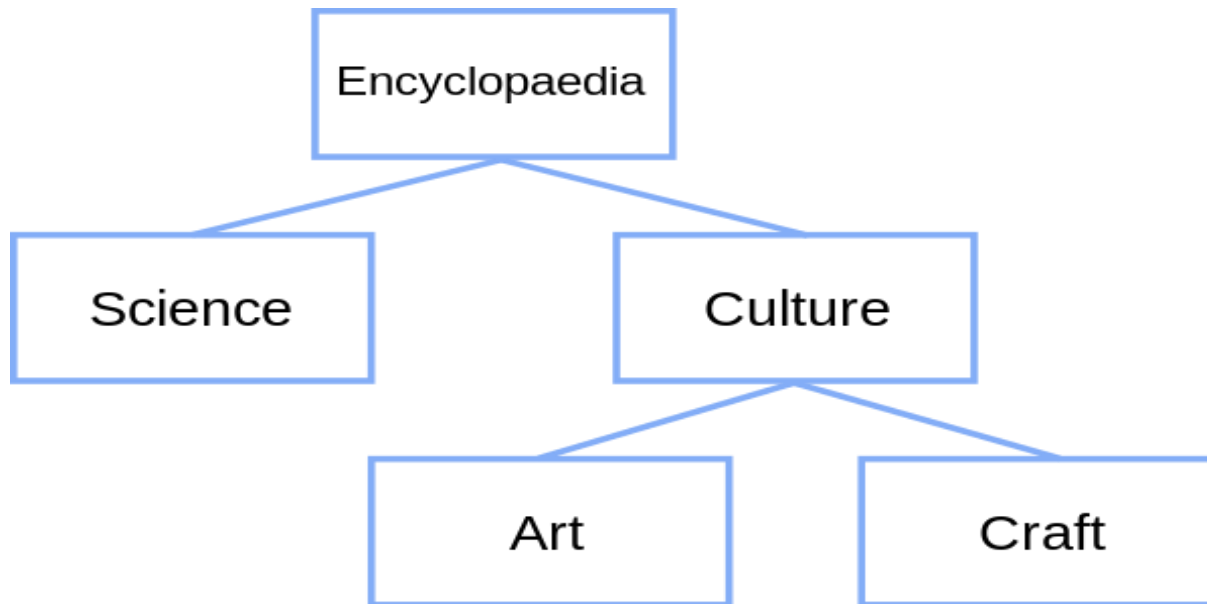
# Read all the text into one string
string_pdf = "\n\n".join(pdf)

# Print the first 500 characters in the string
print("\nThe first 500 symbols in the string:\n", string_pdf[:500])
```

The first 500 symbols in the string:
Think Python
How to Think Like a Computer Scientist

Web Scraping

- **Download Entire Website:** Recursive download of HTML pages, starting from a root HTML page (using command line tool **wget** or **curl**)
- On the computer, **a website is stored as nested folders** that reflect the website hierarchical structure (e.g., Encyclopedia/Culture/Art)



Web Scraping

- Good practice for web scraping:
 - Check a website's terms and conditions before you scrape their pages
 - Do not target a website with too many automated requests, this can slow or bring their website server down (denial of service attack; use a delay between requests, eg request a page then wait 5 seconds)
 - Websites often change their layout, so scrapers often break and need to be re-written (that is why libraries like **newspaper** are so popular, as they can work with different html formats)

Scraping of website using wget

#Download all the newposts from the Insight website recursively:

```
wget -r -E -p -l 1 https://www.insight-centre.org/news/
```

Ctrl + C to stop downloading

To check all parameters for wget run: **wget - -help**

- | | | |
|-----------|--------------------|--|
| -r | --recursive | specify recursive download |
| -E | --adjust-extension | save HTML/CSS documents with proper file extensions |
| -p | --page-requisites | get all images, etc. needed to display the HTML page |
| -l | --level=number | maximum recursion depth |

Can be used to: download books from Project Gutenberg, free books from Kdnuggets, news articles from news websites, backup entire websites, etc.

Web APIs

- Instead of manually scraping HTML data, some websites provide a convenient, official, way of retrieving their data, aka API.
- **Web API** (“Web Application Programming Interface”): Set of HTTP request messages + definition of the structure of the response
- **Formats:** API calls can return a variety of formats: HTML, XML, **JSON**
- **Rate Limits:** Most APIs have rate limits, i.e., we can only make a certain number of calls to the API from a given IP address or account, in each amount of time, e.g., 10 requests per minute.
- **Authentication:** Some APIs require authentication (i.e., need to create an account and get authentication tokens/passwords)

Web APIs

- Getting to the data:
 - Using a URL, also called **API endpoint**
 - Using a **programming language** (e.g., Python libraries written for that specific API) to request data

Web APIs

- **Example API access using an API endpoint URL:**
 - Public database online:
<https://data.colorado.gov/Business/Colorado-Business-Entities/4ykn-tg5h>
 - Use available API to filter, query, aggregate data
 - API Endpoint: <https://data.colorado.gov/resource/4ykn-tg5h.json>
 - To only get subset of data, use modified URL:
<http://data.colorado.gov/resource/4ykn-tg5h.json?principalCity=Denver>

Databases

- **Relational SQL Databases:** Organise data into one or more tables and describe relations between them (e.g., SQLite, MySQL, PostgreSQL)
- **NoSQL Databases:**
 - Provide a mechanism for storage and retrieval which is modeled differently than relational databases, e.g., using data structures such as: key-value (json), graph, or document
 - Increasingly used in big data and real-time web applications (e.g., MongoDB)
- We often need to store data in a database for later access. If you work with data, it is useful to be comfortable with relational databases and the SQL language

Practicing Getting Data

- Python notebook gives examples of how to get data from the Web, in different formats

[Lab2-Notebook-GettingData.ipynb](#)

[Week2-Lab2-Tasks.txt](#) gives guidance on tasks to do during the lab.

Check out the [Discussion forum on Brightspace](#).

API Resources

Sample Public APIs:

- OpenWeatherMap API - <https://openweathermap.org/api>
- OpenStreetMap API - <https://www.openstreetmap.org>
- Foursquare API - <https://developer.foursquare.com/>
- CSO API: <https://data.cso.ie>
- World Bank APIs - <http://data.worldbank.org/developers>
- Guardian Content API - <http://open-platform.theguardian.com/>
- New York Times Developer Network - <http://developer.nytimes.com/>
- Reddit API - <https://www.reddit.com/dev/api/>

Sample Web API Lists:

- https://en.wikipedia.org/wiki/List_of_open_APIs
- <https://github.com/toddmotto/public-apis>
- <http://www.programmableweb.com/apis/directory>
- <http://www.computersciencezone.org/50-most-useful-apis-for-developers>

References

- Web:
 - <http://www.nltk.org/book/>
 - <https://unix.stackexchange.com/questions/47434/what-is-the-difference-between-curl-and-wget>